Proiectarea Algoritmilor

Curs 4 – Backtracking şi propagarea restricţiilor



Bibliografie

- http://ktiml.mff.cuni.cz/~bartak/constraints/intro.html
- https://algorithm-visualizer.org/
- Sudoku – Backtracking: https://gist.github.com/johnthebrave/229f28a66efc71bd9a72bb79e7b66a58
- Sudoku – AC1: https://gist.github.com/johnthebrave/266ac941d5cba287235ced3ec9d89a7e
- NQueens: https://gist.github.com/johnthebrave/dfc43837aa466dbf70912d5a833f460c

Problema

	2		8	1		7	4	
7					3	1		
	9				2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

SUDOKU

- Joc foarte la modă cu reguli foarte simple.
- Fiecare rând, coloană sau regiune nu trebuie să conţină decât o dată cifrele de la unu la nouă (Wikipedia).
- Prin trecerea în revistă a soluţiilor posibile pentru acest joc vom explora tehnicile de rezolvare backtracking şi propagarea restricţiilor.



Soluţia 1 – generează şi testează

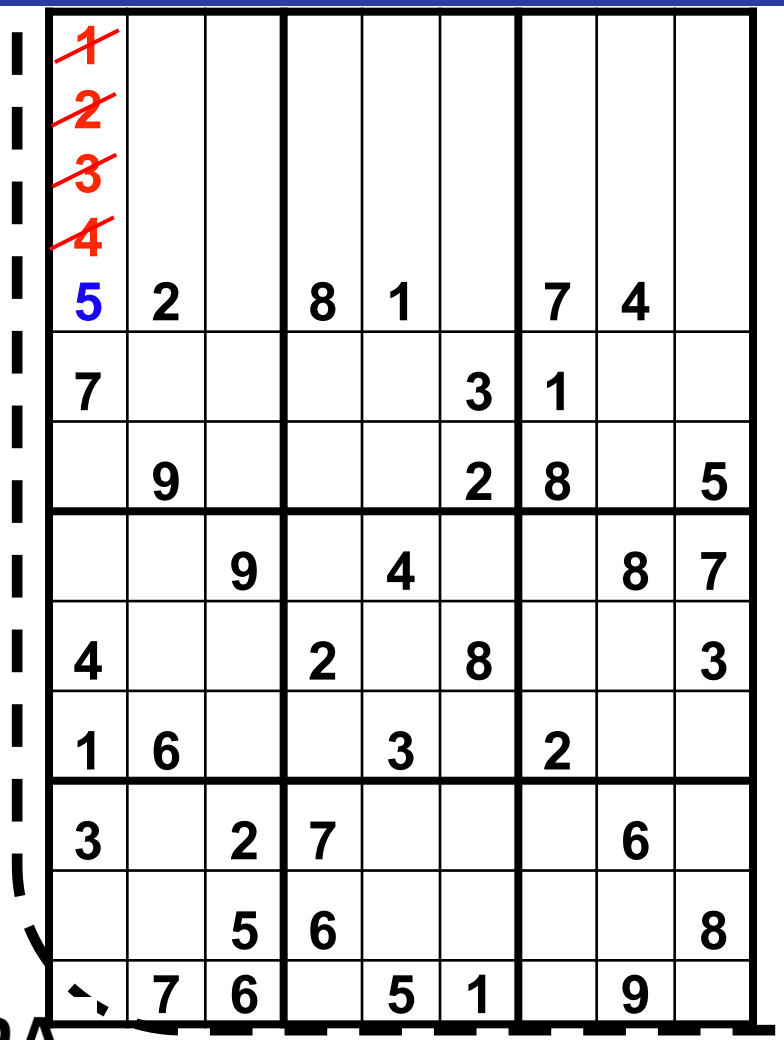
- Generăm toate soluţiile posibile şi le testăm.
- 45 spaţii de completat, 9 posibilităţi de completare pentru fiecare căsuţă => 9^{45} soluţii de testat.

1-9	2	1- 9	8	1	1-9	7	4	1-9
7	1-9	1-9	1-9	1-9	3	1	1-9	1-9
1-9	9	1-9	1-9	1-9	2	8	1-9	5
1-9	1-9	9	1-9	4	1-9	1-9	8	7
4	1-9	1-9	2	1-9	8	1-9	1-9	3
1	6	1-9	1-9	3	1-9	2	1-9	1-9
3	1-9	2	7	1-9	1-9	1-9	6	1-9
1-9	1-9	5	6	1-9	1-9	1-9	1-9	8
1-9	7	6	1-9	5	1	1-9	9	1-9

Soluţia 2 – Backtracking cronologic (orb) (I)

- Construieşte soluţiile iterativ.
- Menţine evidenţa alegerilor făcute.
- În momentul în care se ajunge la o contradicţie se revine la ultima decizie luată şi se încearcă alegerea unei alte variante.

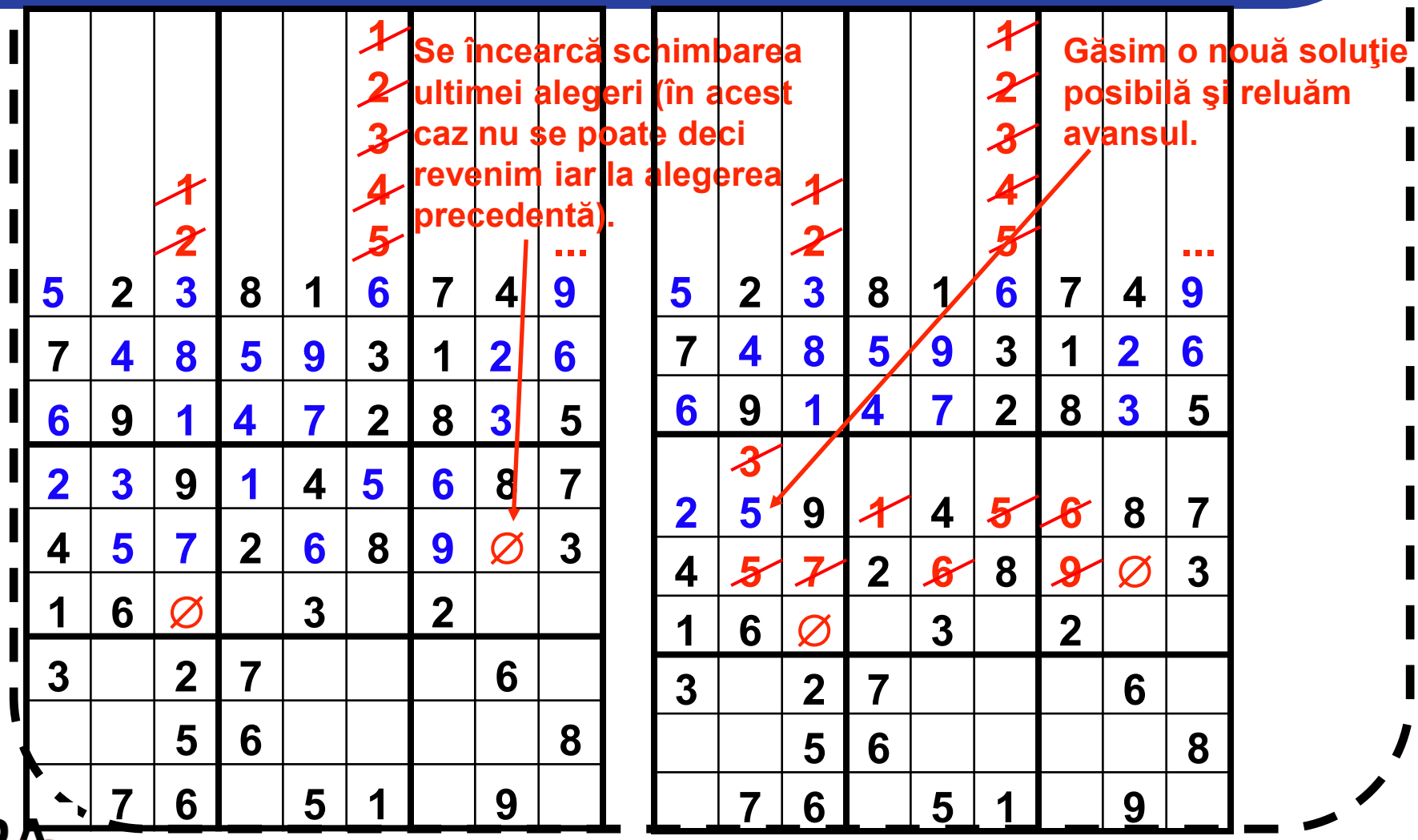
Soluţia 2 – Backtracking cronologic (orb) (II)





Nu se găseşte nici o soluţie pe această cale deci trebuie să revenim.

Soluţia 2 – Backtracking cronologic (orb) (III)



Soluţia 2 – Backtracking cronologic (orb) (IV)

Schema Backtracking

- Soluție-parţială ← INIT // inițializez
- EȘEC-DEFINITIV ← fals // nu am ajuns (încă) la eșec
- **Cât timp** Soluție-parţială nu este soluţie finală **și nu avem** EȘEC-DEFINITIV
  - Soluție-parţială ← AVANS (Soluție-parțială) // avansez
  - **Dacă** EȘEC (Soluție-parțială) // nu mai pot avansa
    - **Atunci** REVENIRE (Soluție-parțială) // mă întorc
- **Dacă** EȘEC-DEFINITIV
  - **Atunci** Întoarce EȘEC // nu s-a găsit nicio soluție
  - **Altfel** Întoarce SUCCES // am ajuns la soluția problemei
- **Sfârșit.**

Procedura AVANS (Soluție-parțială)

- **Dacă** există alternativă de extindere // pot avansa?
  - **Atunci** Soluție-parțială ← Soluție-parțială ∪ alternativă de extindere // avansez
  - **Altfel Dacă** Soluție-parțială este INIT
    - **Atunci** EȘEC-DEFINITIV ← adevărat // nu s-au găsit soluții pentru problemă
    - **Altfel** EȘEC (Soluție-parțială) // ramura curentă a dus la eșec



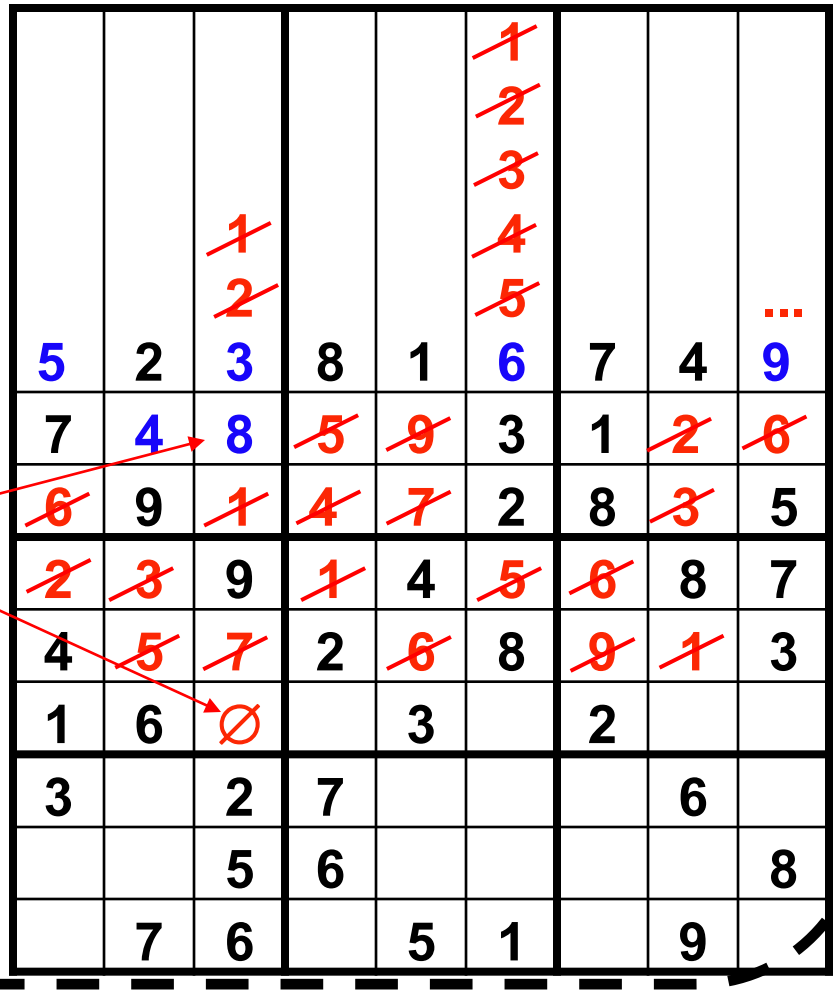
Backtracking – optimizări posibile (I)

- Alegerea variabilelor în altă ordine.
- Îmbunătăţirea revenirilor.
  - Necesită detectarea cauzei producerii erorii.
- Evitarea redundanţelor în spaţiul de căutare (îmbunătăţirea avansului).
  - Evitarea repetării unei căutări care ştim că va duce la un rezultat greşit.

Backtracking – optimizări posibile (II)

Îmbunătăţirea revenirilor

Revenire la alegerea variabilei care a cauzat eşecul (8 nu poate fi pus decât la poziţia indicată).



Backtracking – optimizări posibile (III)

Evitarea redundanţelor în spaţiul de căutare

Alegerea lui 8 pe această poziţie va produce un eşec în viitor indiferent de celelalte alegeri făcute deci în cazul revenirii în această poziţie nu are sens să mai facem această alegere.

		~~1~~ ~~2~~			~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~			...
5	2	3	8	1	6	7	4	9
7	4	~~8~~	~~5~~	~~9~~	3	1	~~2~~	~~6~~
~~6~~	9	~~1~~	~~4~~	~~7~~	2	8	~~3~~	5
~~2~~	~~3~~	9	~~1~~	4	~~5~~	~~6~~	8	7
4	~~5~~	~~7~~	2	~~6~~	8	~~9~~	~~1~~	3
1	6	∅		3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Restricţii, reţele de restricţii, probleme de prelucrarea restricţiilor

- Definiție: O restricţie c este o relaţie între una sau mai multe variabile $v_1, \ldots, v_m$, (denumite nodurile sau celulele restricţiei). Fiecare variabilă v_i poate lua valori într-o anumită mulţime D_i, denumită domeniul ei (ce poate fi finit sau nu, numeric sau nu).

- Definiție: Se spune că un tuplu (o atribuire) de valori $(x_1, \ldots, x_m)$ din domeniile corespunzătoare celor m variabile satisface restricţia $c(v_1, ..., v_m)$, dacă $(x_1, \ldots, x_m) \in c(v_1, ..., v_m)$.

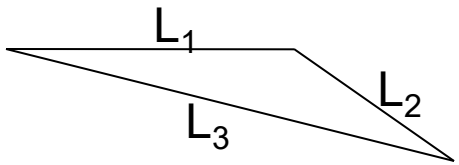
Exprimarea restricţiilor

- Enumerarea tuplurilor restricţiei.
  - **(5,2,3,8,1,6,7,4,9); (6,2,3,8,1,5,7,4,9); etc.**
- Formule matematice, cum ar fi ecuaţiile sau inecuaţiile.
  - $0 < V_{1j} < 10$; $V_{1j} \neq V_{1k}$; $\forall\ j \neq k,\ 0 < j, k < 10$
- Precizarea unei mulţimi de reguli.

	2		8	1		7	4	
7					3	1		
	9				2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Tipuri de restricţii

- Unare
  - Specificarea domeniului variabilei.
  - $V_{11} \in \{1, 2, 3, 4, 5, 6, 7, 8, 9\} \setminus \{2, 8, 1, 7, 4, 3, 9\}$
- Binare
  - Între 2 variabile.
  - $V_{1j} \neq V_{1k}\ \forall j \neq k, 0 < j, k < 10$
- N-are
  - Între n variabile.
  - Regula triunghiului: $L_1 + L_2 > L_3$.



Probleme de satisfacerea restricţiilor

- Definiție: O problemă de satisfacere a restricţiilor (PSR) este un triplet <V,D,C>, format din:
  - o mulţime V formată din n variabile V = $\{v_1, \ldots, v_n\}$;
  - mulţimea D a domeniilor de valori corespunzătoare acestor variabile: D = $\{D_1, \ldots D_n\}$;
  - o mulţime C de restricţii C = $\{c_1, \ldots, c_p\}$ peste submulţimi ale V (c_i (v_{i1}, . . ., v_{ij}) $\subseteq$ D_{i1} x D_{i2} x ... x D_{ij}).

- Conform Definiției tuplului, o restricţie c_i (v_{i1}, ..., v_{ij}), este o submulţime a produsului cartezian D_{i1} x D_{i2} x ... x D_{ij}, constând din toate tuplurile de valori considerate că satisfac restricţia pentru (v_{i1}, . . ., v_{ij}).

Soluții ale PSR

- Definiție: O soluţie a unei PSR <V,D,C> este un tuplu de valori <x_1, … , x_n> care conține toate variabilele V, din domeniile corespunzătoare D, astfel încât toate restricţiile din C să fie satisfăcute.

- Definiție: PSR binară este o PSR ce conţine doar restricţii unare şi binare.

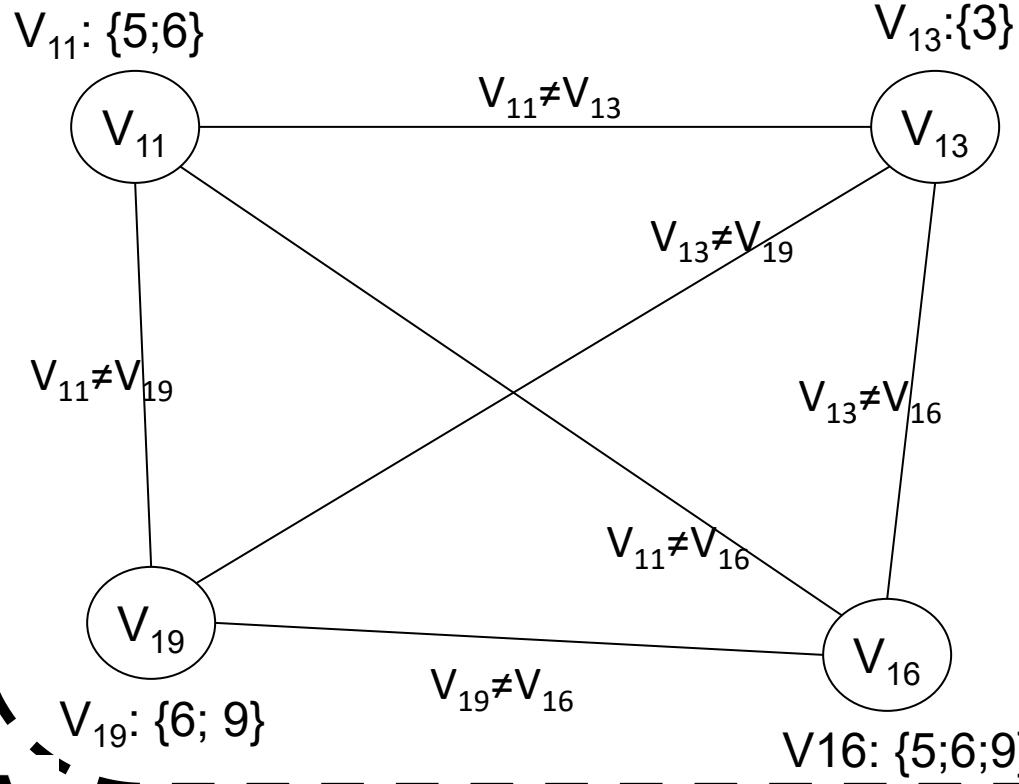
Probleme de satisfacere a restricţiilor - Reprezentare

- Reprezentare PSR prin reţele de restricţii.
- Reprezentarea folosită: graf
  - Nodurile: variabilele restricţiei
  - Arcele: restricţiile problemei
  - Valabilă doar pentru PSR binare
- Altă reprezentare posibilă: graf
  - Nodurile: restricţiile problemei
  - Arcele: variabilele restricţiei
  - Valabilă pentru orice tip de PSR



Exemplu de reprezentare a PSR

- $0 < V_{1j} < 10;\ V_{1j} \neq V_{1k};$
  $\forall j \neq k,\ 0 < j, k < 10$



5;6	2	3	8	1	5;6;9	7	4	6;9
7					3	1		
	9				2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Algoritmi de rezolvare a PSR – Propagarea restricțiilor

- Caracteristici
  - Rezolvă PSR binare;
  - Variabilele au domenii finite de valori;
  - Prin propagarea restricţiilor se filtrează mulţimile de valori (se elimină elementele din domeniu conform unui criteriu dat);
  - Procesul de propagare se opreşte când:
    - O mulţime de valori este vidă → EȘEC;
    - Nu se mai modifică domeniul vreunei mulţimi.



Algoritmi de rezolvare a PSR – Propagarea restricțiilor

Notaţii:

- n = număr variabile = număr restricţii unare;
- r = număr restricţii binare;
- G = reţeaua de restricţii cu variabile drept noduri și restricţii drept arce;
- D_i = domeniul variabilei i;
- Q_i = predicat care verifică restricţia unară pe variabila i;
- P_{ij} = predicatul care reprezintă restricţia binară pe variabilele i și j (O muchie între i și j se înlocuiește cu arcele orientate de la i la j și de la j la i);
- a = max $|D_i|$.

NC-1 (Node Consistency -1)

- Algoritm de consistenţa nodurilor (pentru restricţii unare).

- Procedura NC(i) este:
  - **Pentru fiecare** x $\in$ D_i
    - **Dacă** not Q_i (x) // nu este satisfăcută restricția unară
      - **Atunci** șterge x din D_i
  - **Sfârșit**.

- Algoritm NC-1 este:
  - **Pentru** i **de la** 1 **la** *n* **Execută** NC(i) // pentru fiecare var
  - **Sfârșit**.

Complexitate NC-1? na

NC-1: Exemplu (I)

- Elimină din domeniul de valori al fiecărui nod valorile care nu satisfac restricţiile care au ca argument variabila din nodul respectiv.
- În cazul Sudoku variabilele iniţial iau valori între 1-9.
- Algoritmul NC-1 elimină pentru fiecare variabilă acele valori din domeniu care nu sunt consistente cu valorile fixe (celulele deja fixate).



NC-1: Exemplu (II)

5;6	2	3;	8	1	5;6;9	7	4	6;9;
7	4;5;8;	4;8;			3	1		
6;	9	1;3;4;			2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Algoritmi de consistenţă a arcelor

- Algoritmii de consistenţă a arcelor înlătură toate inconsistenţele submulţimilor de 2 elemente ale reţelei de restricţii.

- Funcţia REVISE ((i,j)) este:
  - ȘTERS ← fals // nu am modificat domeniul de valori
  - **Pentru fiecare** $x \in D_i$
    - **Dacă** nu există $y \in D_j$ a.î. $P_{ij}(x,y)$ // nu se respectă restricția
      - **Atunci**
        - Șterge x din D_i;
        - ȘTERS ← adevărat; // am făcut modificări
  - **Întoarce** ȘTERS.

Exemplu funcţionare REVISE

x

y

~~5;6~~	2	3;	8	1	5;6;9	7	4	6;9;
7	4;5;8;	4;8;			3	1		
6;	9	1;~~3~~;4;			2	8		5
5;6;2;		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
9;		5	6					8
8;	7	6		5	1		9	

Complexitate Revise ? a^2

REVISE - Concluzie

- Funcţia Revise este apelată pentru un arc al grafului de restricţii (binare) și șterge acele valori din domeniul de definiţie al unei variabile pentru care nu este satisfăcută restricţia pentru nici o valoare corespunzătoare celeilalte variabile a restricţiei.

- Complexitate Revise: $O(a^2)$



AC-1 (Arc Consistency -1)

- Algoritm AC-1 este:
  - NC-1; // reduc domeniul de valori
  - Q ← {(i,j) | (i,j) ∈ arce(G), i ≠ j} // adaug restricțiile
  - **Repetă**
    - SCHIMBAT ← fals // nu am modificat niciun // domeniu
    - **Pentru fiecare** (i,j) ∈ Q // pentru fiecare restricție
      - SCHIMBAT ← (REVISE ((i,j)) sau SCHIMBAT)
  - **Până când *non*** SCHIMBAT // nu am mai făcut // modificări
  - **Sfârșit.**

AC-1 Caracteristici & Complexitate

- Se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare.

- Complexitate: $O(na * 2r * a^2)$

La fiecare iteraţie (din Repetă) eliminăm o singură valoare (şi avem maxim na valori posibile).

Numărul de apelări al Revise într-o iteraţie (din Repetă).

Complexitate Revise.

Exemplu AC-1

~~5,6~~	2	3;	8	1	5;6;9	7	4	6;9;
7	4;5;8;	4;8;			3	1		
6;	9	1;~~3~~;4;			2	8		5
~~5;6;2;~~	~~3,5~~	9		4			8	7
4	5	7	2		8			3
1	6	8		3		2		
3		2	7				6	
9;		5	6					8
8;	7	6		5	1		9	

AC-3 (Arc Consistency -3)

- Algoritm AC-3 este:
  - NC-1; // reduc domeniul de valori
  - Q ← {(i,j) | (i,j) ∈ arce(G), i ≠ j} // adaug // restricțiile
  - **Cât timp** Q nevid
    - Selectează și șterge un arc (k,m) din Q;
    - **Dacă** REVISE ((k,m)) // am modificat domeniul
      - **Atunci** Q ← Q ∪ {(i,k) | (i,k) ∈ arce(G), i ≠ k, i ≠ m} // verific dacă nu se modifică și alte domenii

AC-3 Caracteristici

- Se elimină pe rând arcele (constrângerile).
- Dacă o constrângere aduce modificări în reţea adăugăm pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate.
  - Scopul: Reverificarea nodurilor direct implicate de o constrângere din reţea.
- Avantaj: Se fac mult mai puţine apeluri ale funcţiei REVISE.
- Complexitate: $O(a^3r)$.

Backtracking + Propagarea restricţiilor

- În general, propagarea restricţiilor nu poate rezolva complet problema dată.
- Metoda ajută la limitarea spaţiului de căutare (foarte importantă în condiţiile în care backtracking-ul are complexitate exponenţială!).
- În cazul în care propagarea restricţiilor nu rezolvă problema se foloseşte:
  - Backtracking pentru a genera soluţii parţiale;
  - Propagarea restricţiilor după fiecare pas de backtracking pentru a limita spaţiul de căutare (şi eventual a găsi că soluţia nu este validă).

ÎNTREBĂRI?



Bibliografie Curs 5

- Giumale – Introducere in Analiza Algoritmilor cap 7.6
- http://www.dwheeler.com/chess-openings/#Sicilian%20Defense
- http://mouserunner.com/MozllaTicTacToe/Mozilla_Tic_Tac_Toe.htm
- http://www.emunix.emich.edu/~evett/AI/AlphaBeta_movie/index_movie.htm